1ÈRE NSI · FICHE RÉSUMÉ

Les bases *de Python*

1 Variables et affectation

- Un **programme** est une suite d'**instructions** exécutées **de haut en bas**, une par une. Python s'arrête **à la ligne de l'erreur**.
- Une **variable** est un **nom** associé à une **valeur** (une étiquette).
- `nom = valeur` : Python **calcule d'abord la droite**, puis affecte.
- `=` **affecte** ; `==` **compare**.

```
pv = 100
pv = pv - 30   # « pv prend la valeur pv − 30 »
```

NOMMER UNE VARIABLE

- Lettres **sans accent**, chiffres, `_`. Pas de chiffre au début, **ni espace ni tiret**, pas de mot-clé (`if`, `for`…).
- `nom` ≠ `Nom` : Python distingue les majuscules.

CORRECT	INCORRECT
`nombre_de_pv`, `pv_max`, `niveau2`	`2eme_vie`, `mon nom`, `prix-total`, `if`

2 Les types

TYPE	NOM	EXEMPLES
`int`	entier	`5`, `-3`, `7 // 2`
`float`	flottant (décimal)	`3.14`, `6.0`, `7 / 2`
`str`	chaîne de caractères	`"Naruto"`, `'7'`
`bool`	booléen	`True`, `False`, `7 > 2`

- `type(x)` donne le type. `"7"` entre guillemets est une **chaîne**, pas un nombre.
- Conversions : `int("16")` → `16` · `float("6.9")` → `6.9` · `str(42)` → `"42"` · `int(3.9)` → `3` (tronqué).

> `int("abc")` → **ValueError** : la conversion doit être possible.

3 Opérateurs

OPÉRATEUR	RÔLE	EXEMPLE
`+ - *`	priorités des maths	`2 + 3 * 4` → `14`
`/`	division (→ **float**)	`7 / 2` → `3.5`
`//`	quotient euclidien	`17 // 5` → `3`
`%`	reste euclidien	`17 % 5` → `2`
`**`	puissance	`2 ** 3` → `8`

> `n % 2 == 0` signifie « n est **pair** ».

CHAÎNES

- `+` **concatène** : `"3" + "4"` → `"34"` · `*` **duplique** : `"ha" * 3` → `"hahaha"`.
- `"PV : " + 70` → **TypeError**. Écrire `"PV : " + str(70)`.

COMPARAISONS ET LOGIQUE

- `== != < <= > >=` produisent un **booléen**.
- `A and B` : vrai si **les deux** sont vrais · `A or B` : vrai si **au moins un** l'est · `not A` : **inverse**.

A	B	A AND B	A OR B	NOT A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

4 Afficher et saisir

- `print(a, b)` affiche les valeurs séparées par un **espace**. Il **ne renvoie rien** et ne modifie aucune variable.
- `input("question")` renvoie **toujours une chaîne** (`str`), même si on tape un nombre.

```
nom = input("Nom du manga ? ")
prix = float(input("Prix ? "))
print(nom, ":", prix * 2, "€ les 2 tomes")
```

> Des chiffres « collés » (`"5" + "6"` → `56`) : il manque `int(...)` !

5 Les conditions

```
if temperature > 30:
    print("Short")
elif temperature > 10:
    print("Pull")
else:
    print("Manteau")
print("Fin")  # toujours
```

- **Deux-points** `:` après la condition.
- Le bloc est **indenté** de 4 espaces : c'est l'indentation qui délimite le bloc.
- Les conditions sont testées **dans l'ordre** : seul le **premier bloc vrai** s'exécute.

DEUX `IF` À LA SUITE	`IF … ELIF`
tests **indépendants** : plusieurs blocs peuvent s'exécuter	tests **exclusifs** : un seul bloc s'exécute

Conditions composées : `if age >= 12 and age < 18:`

Je sais…

- ☐ nommer une variable et faire une affectation
- ☐ donner le type d'une valeur et la convertir
- ☐ utiliser `/` `//` `%` `**` et les opérateurs sur les chaînes
- ☐ évaluer une expression avec `and`, `or`, `not`
- ☐ afficher avec `print` et saisir avec `int(input(...))`
- ☐ écrire un `if / elif / else` et prévoir quel bloc s'exécute
- ☐ définir une fonction, l'appeler et récupérer son résultat
- ☐ expliquer la différence entre `return` et `print`
- ☐ donner les valeurs de `range` à 1, 2 ou 3 paramètres
- ☐ choisir entre `for` et `while`, éviter une boucle infinie
- ☐ faire le tableau de suivi d'un programme
- ☐ lire un message d'erreur (type et ligne)

6 Les fonctions

```
def aire_rectangle(longueur, largeur):
    return longueur * largeur

resultat = aire_rectangle(4, 3)  # → 12
```

- `def` **définit** la fonction : son code ne s'exécute qu'à l'**appel**.
- **Paramètres** : les noms dans `def`. **Arguments** : les valeurs données à l'appel.
- `return` **renvoie** une valeur et **termine** la fonction. Sans `return`, la fonction renvoie `None`.

RETURN	PRINT
renvoie une valeur au programme	affiche une valeur à l'écran
valeur stockable, réutilisable	valeur perdue pour le programme
termine la fonction	ne termine rien

VARIABLES LOCALES

Une variable créée dans la fonction n'existe **que pendant l'appel**. Pour garder le résultat, il faut le **récupérer** :

```
perdre(pv, 30)        # ✗ résultat perdu, pv inchangé
pv = perdre(pv, 30)   # ✓
```

LE MODULE RANDOM

```
import random    # une fois, en haut
de = random.randint(1, 6)  # 1 à 6 INCLUS
x = random.random()      # dans [0 ; 1[
ennemi = random.choice(["gobelin", "troll"])
```

> Tester une fonction : un **cas ordinaire** et les **cas frontières** (`est_majeur(18)` et `est_majeur(17)`).

! Les pièges classiques

- `=` pour affecter, `==` pour comparer.
- `input()` renvoie une chaîne : penser à `int(...)` ou `float(...)`.
- `/` donne toujours un `float` : `10 / 5` → `2.0`.
- `range(1, 4)` s'arrête à **3** ; `randint(1, 6)` peut donner **6**.
- Oublier `:` ou l'indentation ; une instruction mal décalée sort du bloc.
- `print` dans une fonction n'est pas `return` : le résultat vaut `None`.
- Appeler `f(x)` sans ranger le résultat : écrire `x = f(x)`.
- `while` : la variable doit exister avant et changer dans la boucle.
- `"Python" == "python"` vaut `False`.

7 Les boucles

FOR ET RANGE

ÉCRITURE	VALEURS DE I	TOURS
`range(5)`	0, 1, 2, 3, 4	5
`range(2, 6)`	2, 3, 4, 5	4
`range(1, 10, 3)`	1, 4, 7	3
`range(10, 0, -2)`	10, 8, 6, 4, 2	5

> La borne de fin n'est **jamais atteinte** : `range(n)` fait **n tours**, de 0 à n − 1.

- `for lettre in "naruto":` parcourt chaque élément d'une chaîne ou d'une liste.
- **Compteur / accumulateur** : initialisé **avant**, mis à jour **dans**, affiché **après** la boucle.

WHILE

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

```
code = ""   # doit exister
while code != "JURASSIC":
    code = input("Code ? ")
```

- Répète **tant que** la condition est vraie, testée **avant** chaque tour.
- Une variable de la condition doit **changer dans la boucle**, sinon : **boucle infinie**.
- `break` sort immédiatement de la boucle.

	FOR	WHILE
Quand ?	nombre de tours **connu**	tant qu'une condition est vraie
Exemples	10 rencontres ; chaque lettre	saisie à redemander ; combat
Mise à jour	automatique	à écrire soi-même

8 Comprendre son programme

LE TABLEAU DE SUIVI : « JOUER LA MACHINE »

```
distance = 60
tours = 0
while distance >= 50:
    distance = distance - 8
    tours = tours + 1
print(distance, tours)
```

ÉTAPE	DIST.	TOURS	≥50 ?
avant	60	0	True
tour 1	52	1	True
tour 2	44	2	False

Une **ligne par tour**, une **colonne par variable**. Affichage : `44 2`.

LIRE UN MESSAGE D'ERREUR (TYPE + LIGNE)

ERREUR	SIGNIFICATION	EXEMPLE
SyntaxError	oubli de `:`, guillemet, parenthèse	`if pv > 0`
IndentationError	décalage incorrect après `:`	bloc non décalé
NameError	nom inconnu, faute de frappe	`print(Nom)`
TypeError	types incompatibles	`"16" + 1`
ValueError	valeur impossible	`int("abc")`
IndexError	indice hors de la liste	`[1, 2][2]`